

# FastComposer-mini

Aldo Acevedo

Based on the FastComposer paper by  
Guangxuan Xiao, Tianwei Yin, William T. Freeman,  
Frédéric Durand, Song Han

**NOTE:** currently = at the time of publication of  
FastComposer (2023-05-21)

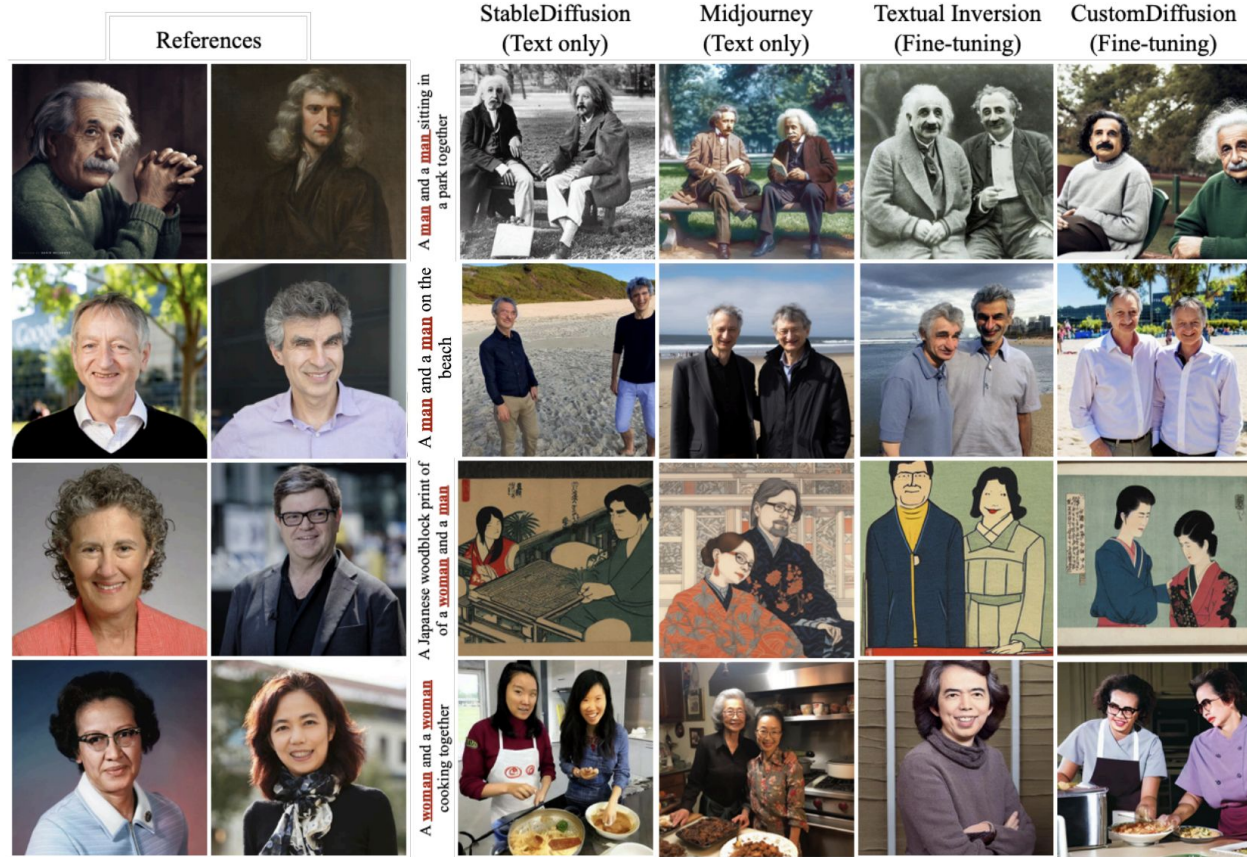
## Three problems with subject-specific image generation (using diffusion models)

1. Fine-tuning the model for each subject is inefficient and costly (and makes multi-subject generation much harder)
2. Identity blending on multi-subject image generation
3. Subject overfitting

# Problem 1: Fine-tuning is costly and inefficient

- Current methods of subject generation involve fine-tuning an existing diffusion model (e.g. Stable Diffusion) to produce the desired subject.

# Problem 2: Identity blending



Source: FastComposer, Xiao et al (2023)

## Problem 3: Subject overfitting

“A woman riding a horse”



Input



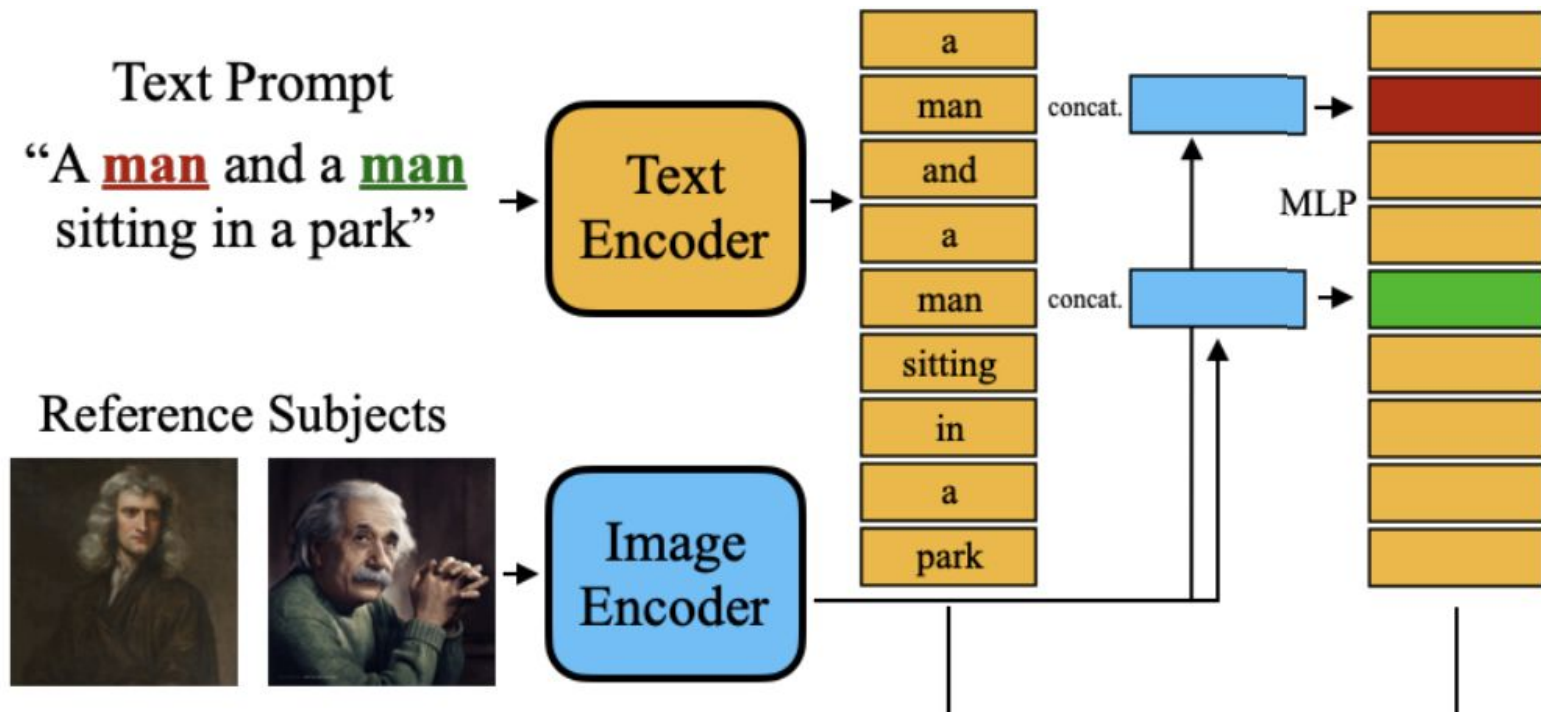
w/o delayed subject  
conditioning

Source: FastComposer, Xiao et al (2023)

# The solutions

1. **Fine-tuning** → Subject-conditional embedding
2. **Identity blending** → Cross-attention localization
3. **Subject overfitting** → Delayed subject conditioning

# Solution to fine-tuning: Input augmentation



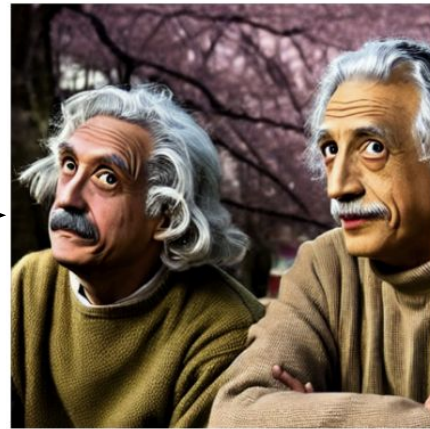
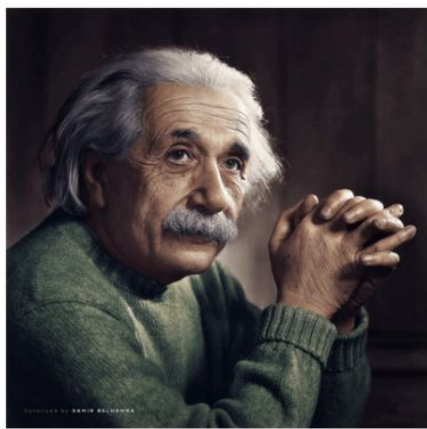
Source: FastComposer, Xiao et al (2023)



But this creates a **problem...**

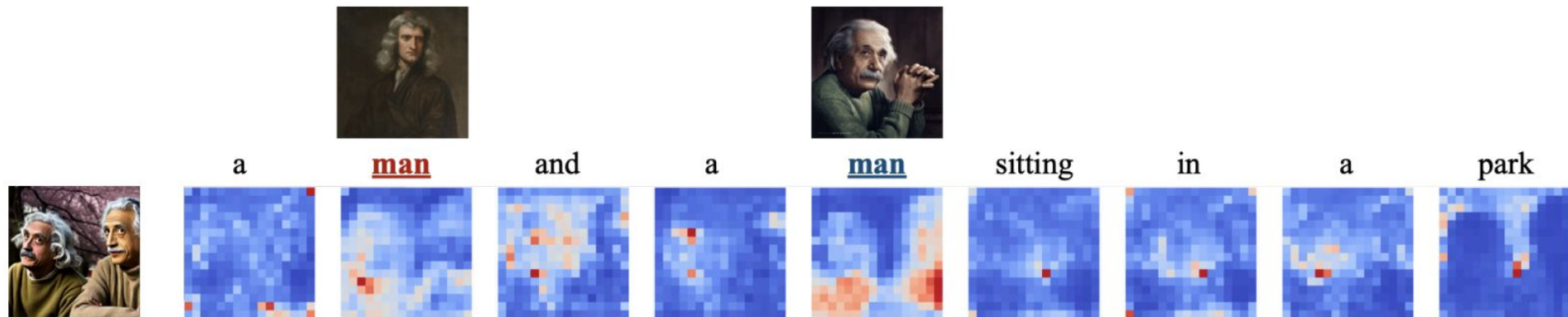
# Going back to identity blending

“A man and a man sitting in a park’



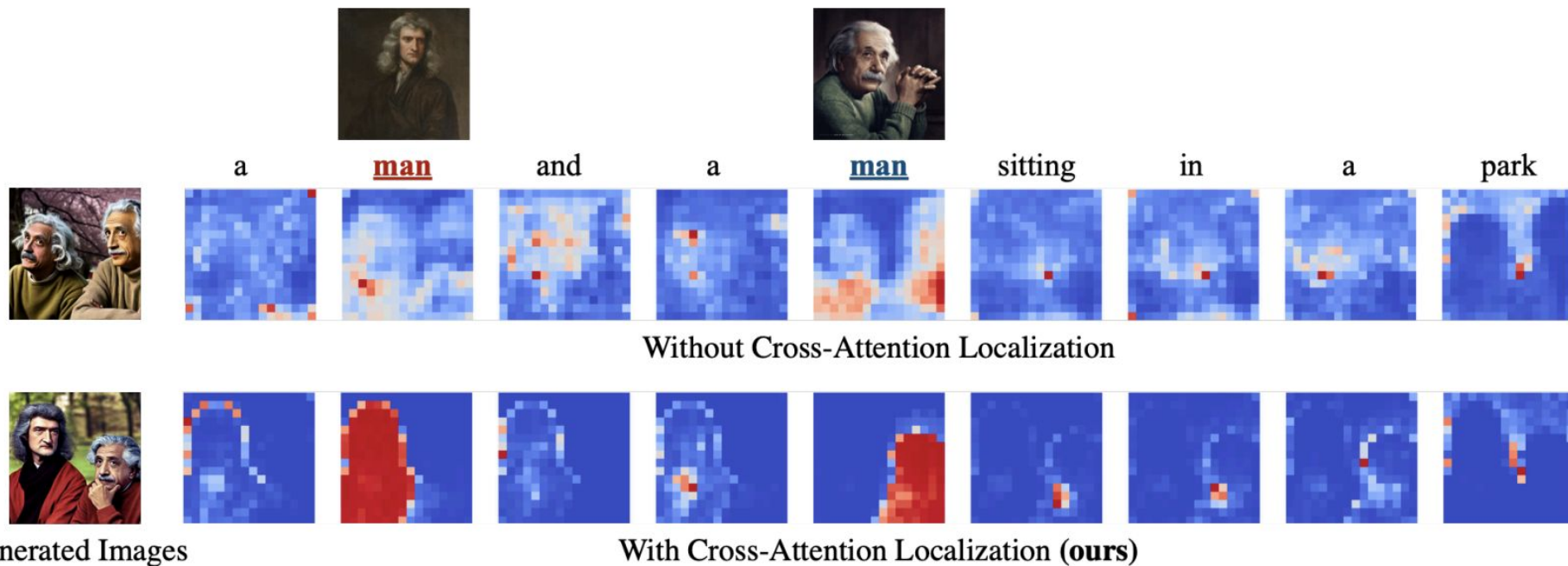
Inputs

Source: FastComposer, Xiao et al (2023)



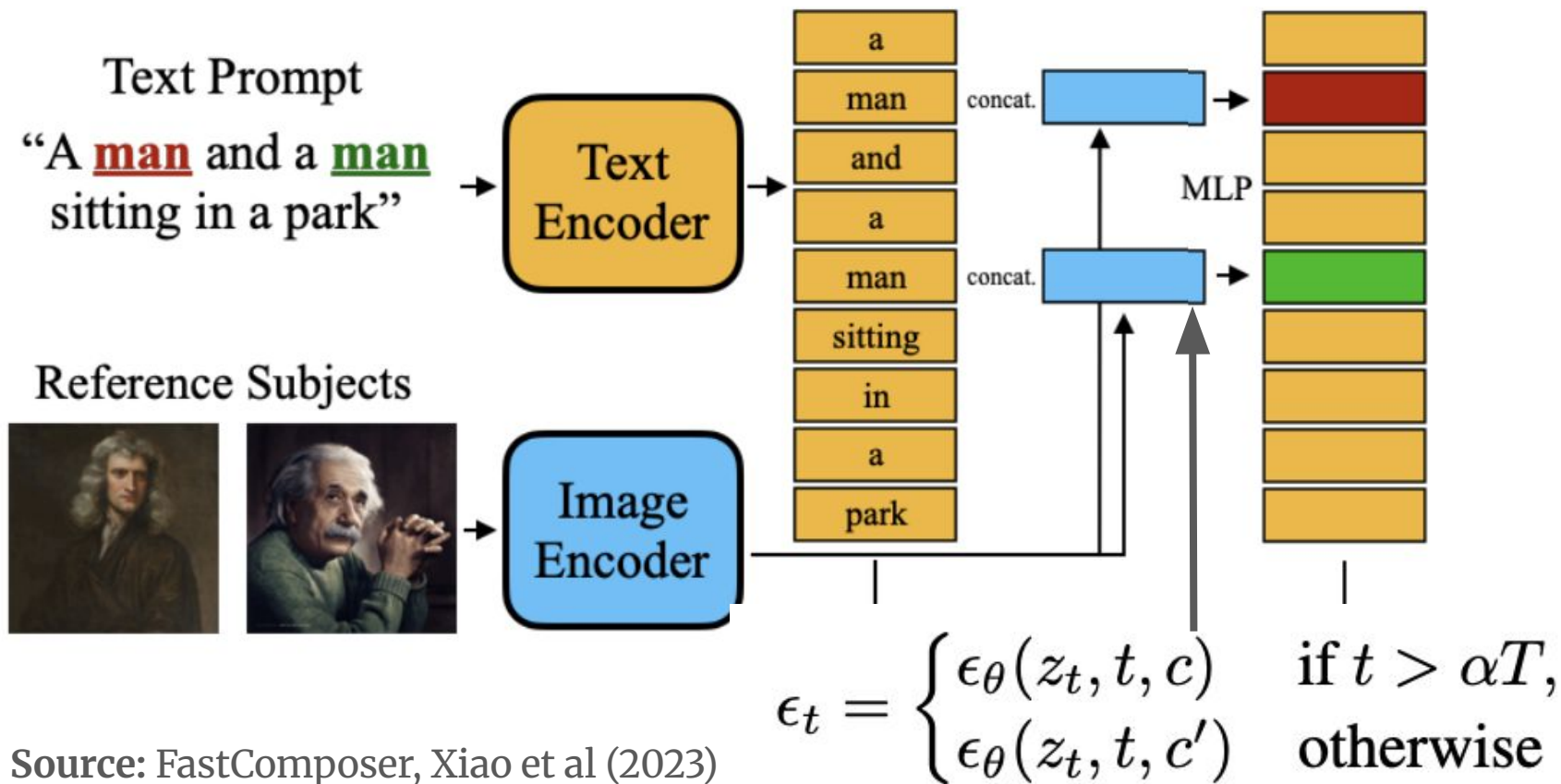
Source: FastComposer, Xiao et al (2023)

# Solution to identity blending: Cross-attention localization

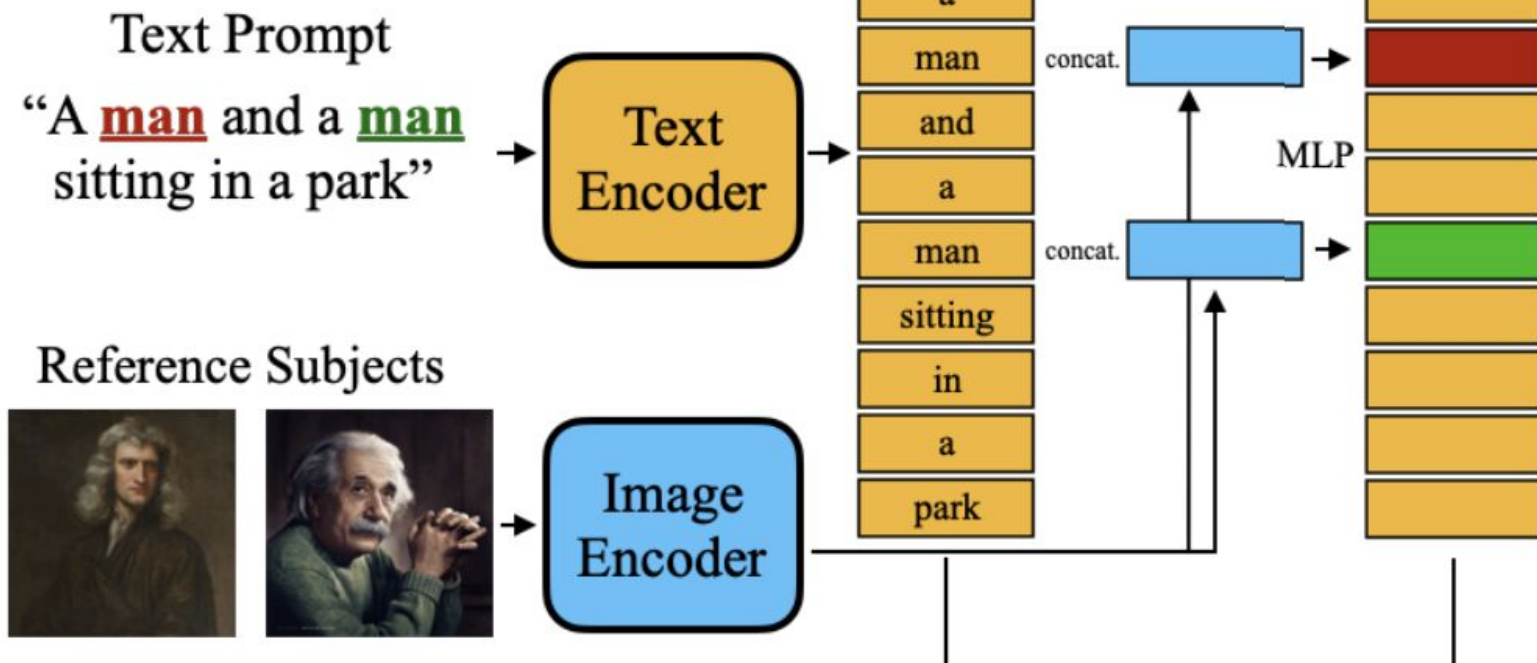


Source: FastComposer, Xiao et al (2023)

# Solution to subject overfitting: delayed subject conditioning

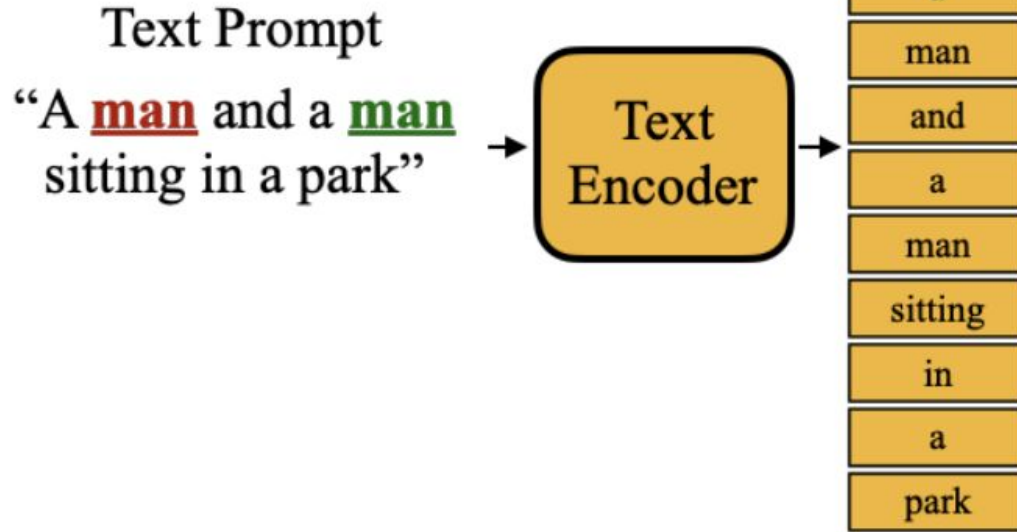


$$\text{if } t > \alpha T$$



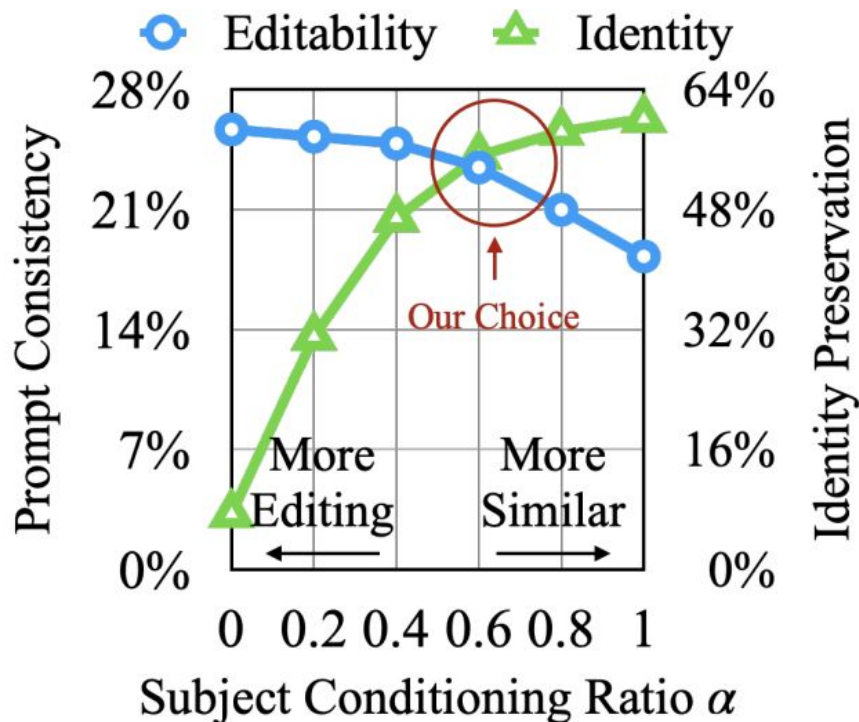
Source: FastComposer, Xiao et al (2023)

# otherwise



Source: FastComposer, Xiao et al (2023)

# Solution to subject overfitting: **delayed subject conditioning**



Source: FastComposer, Xiao et al (2023)



# Solution to subject overfitting: delayed subject conditioning

“A painting of a man and a man in the style of Vincent van Gogh”



“A woman riding a horse”



$\alpha = 0$

$\alpha = 0.2$

$\alpha = 0.4$

$\alpha = 0.6$

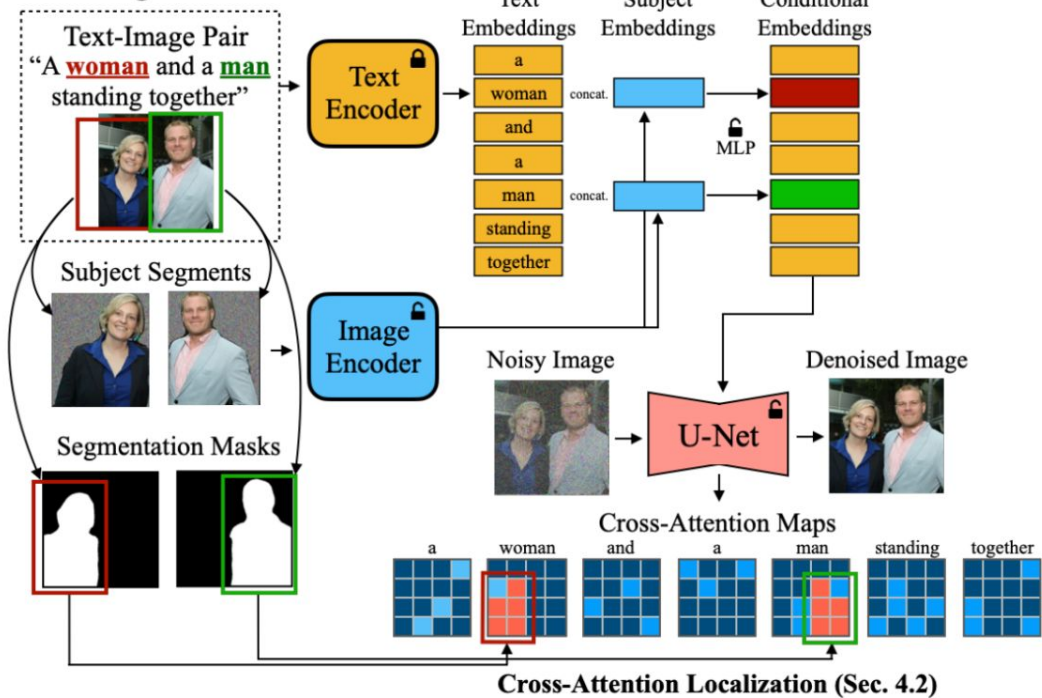
$\alpha = 0.8$

$\alpha = 1.0$

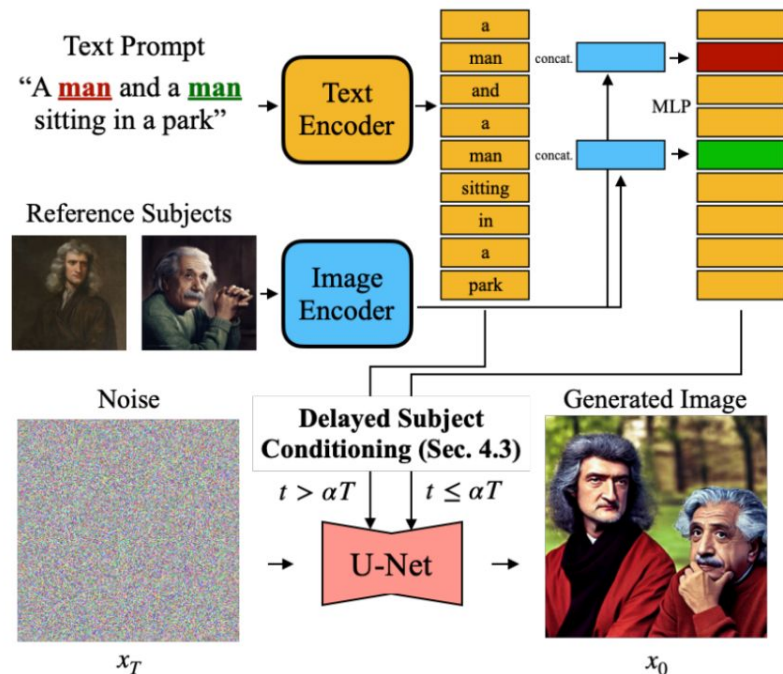
Subject Conditioning Ratio  $\alpha$

# Full FastComposer model overview

## Training



## Inference



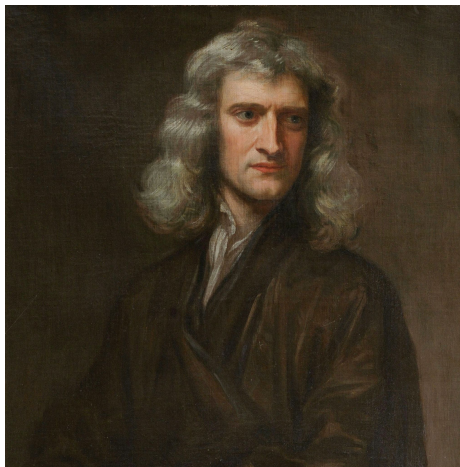
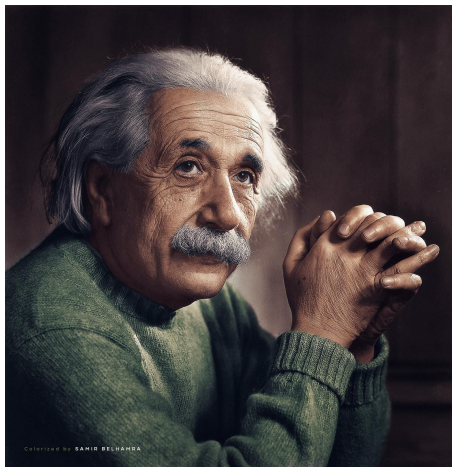
Source: FastComposer, Xiao et al (2023)

# Inputs

Prompt:

a man <|image|> and a man <|image|> are reading book together

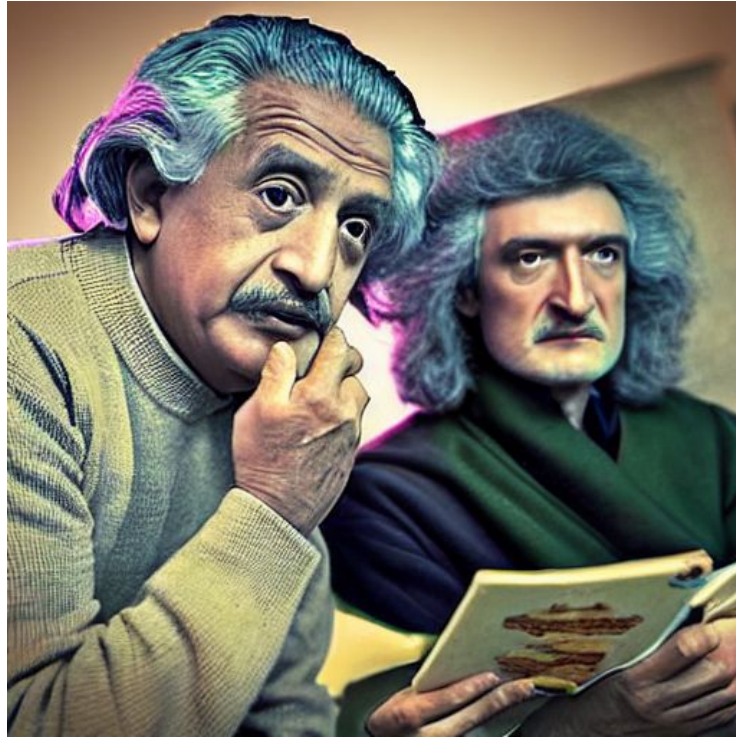
Images:



Source: FastComposer, Xiao et al (2023)



# Output



Source: my laptop

Implementation

# Implementation

- **Backbone (VAE, U-Net):** from `runwayml/stable-diffusion-v1-5`
- **Text/image encoder:** from CLIP
- Subject conditioning is simply a **CLIP image encoder + MLP** on the concatenated tokens
- Delayed subject conditioning is just an **if statement in the denoising loop**
- But how do we do **cross-attention localization**?

## Cross-attention localization implementation

The subject segmentation maps are provided in the dataset. (They were calculated using a panoptic segmentation model.)

To incentivize the model to perform cross-attention localization, we include an additional term in the loss.

$$\mathcal{L}_{\text{loc}} = \frac{1}{m} \sum_{j=1}^m (\text{mean}(A_{i_j} [\bar{m}_j]) - \text{mean}(A_{i_j} [m_j]))$$

$$\mathcal{L}_{\text{loc}} = \frac{1}{m} \sum_{j=1}^m (\text{mean}(A_{i_j} [\bar{m}_j]) - \text{mean}(A_{i_j} [m_j]))$$

Mean attention in parts OUTSIDE the mask

Mean attention in parts INSIDE the mask

Mean over all  $m$  affected attention heads

Affected = middle U-Net layers

$$\mathcal{L} = \mathcal{L}_{\text{noise}} + \lambda \mathcal{L}_{\text{loc}}$$



# Git repo overview

Data and output folders are excluded from the tree shown here.

Source: <https://github.com/mit-han-lab/fastcomposer>

```
— cog.yaml
— demo
  — pipeline.py
  — run_gradio.py
— evaluation
  — __init__.py
  — clip_eval.py
  — single_object
    — __init__.py
    — data.py
    — run.py
    — single_object_evaluation.py
— fastcomposer
  — __init__.py
  — data.py
  — inference.py
  — model.py
  — pipeline.py
  — train.py
  — transforms.py
  — utils.py
— figures
  — multi-subject.png
— LICENSE
— predict.py
— README.md
— scripts
  — run_inference.sh
  — run_training.sh
— setup.py
```

# Git repo overview

Scripts were provided.

Source:  
<https://github.com/mit-han-lab/fastcomposer>

## Download the Pre-trained Models

```
mkdir -p model/fastcomposer ; cd model/fastcomposer  
wget https://huggingface.co/mit-han-lab/fastcomposer/resolve/main/pytorch_model.bin
```

## Inference

```
bash scripts/run_inference.sh
```

## Evaluation

```
python evaluation/single_object/run.py --finetuned_model_path model/fastcomposer/pyt  
python evaluation/single_object/single_object_evaluation.py --prediction_folder OUTP
```

## Training

Prepare the FFHQ training data:

```
cd data  
wget https://huggingface.co/datasets/mit-han-lab/ffhq-fastcomposer/resolve/main/ffhq_  
tar -xvzf ffhq_fastcomposer.tgz
```

Run training:

```
bash scripts/run_training.sh
```

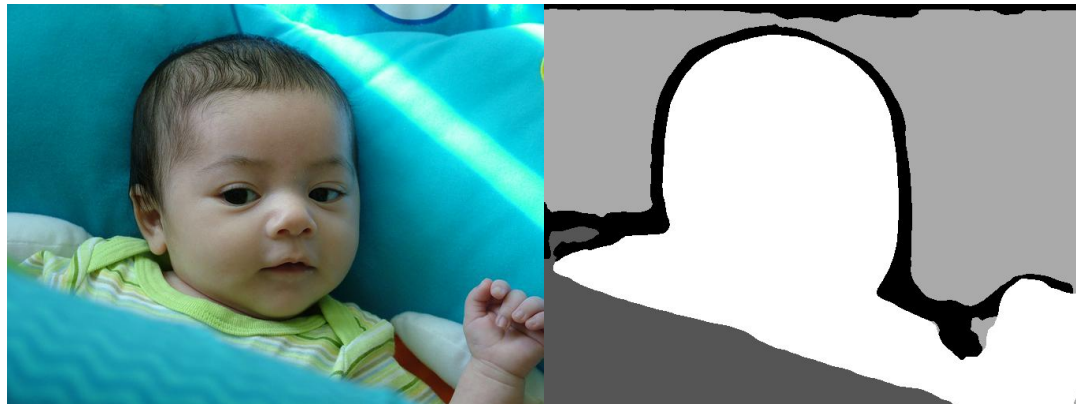
# Datasets

- Training dataset: modified version of FFHQ-wild
  - 70000 image-caption-mask pairs
- Evaluation dataset (single-subject): Celeb-A dataset
  - 15 images of celebrities. These are used to generate 4 images per caption, 40 captions being used.
- Evaluation dataset (multi-subject): not provided in repository

## Source:

- <https://huggingface.co/datasets/mit-han-lab/ffhq-fastcomposer>
- <https://github.com/mit-han-lab/fastcomposer>

# Training dataset example



```
{  
  ...  
  "caption": "a baby laying on a blue blanket with a blue pillow"  
  ...  
}
```

Source: <https://huggingface.co/datasets/mit-han-lab/ffhq-fastcomposer>

# Single subject evaluation dataset



Captions:

a man wearing a red hat

a man wearing a santa hat

a man wearing a rainbow scarf

a man wearing a black top hat and a monocle

Source: <https://github.com/mit-han-lab/fastcomposer>

How can we improve this?

# How can we improve this?

Avenues:

1. **Change backbone.** Currently using StableDiffusion-v1-5. **Possibilities:** image quality gain and speed-ups.
2. **Change attention mechanism.** Currently it uses vanilla attention and scaled dot-product attention. **Possibilities:** speed-ups.
3. **Generalize usage.** Currently, we must manually specify which tokens to augment (with the images) in the prompt. A more general version could have its own mechanism to determine which tokens to augment. (An attention mechanism could be used for this.) **Possibilities:** more ease of use and increased model's latent knowledge.

# Change backbone

Change runwayml/stable-diffusion-v1-5 for segmind/tiny-sd

## SD-Pytorch original model - A100 80GB

```
Loading pipeline components...: 100%| 6/6 [00:01<00:00, 4.82it/s]
100%| 100/100 [00:03<00:00, 28.92it/s]
100%| 100/100 [00:02<00:00, 36.39it/s]
100%| 100/100 [00:02<00:00, 37.08it/s]
100%| 100/100 [00:02<00:00, 36.77it/s]
100%| 100/100 [00:02<00:00, 36.38it/s]
```

## SD Tiny model - A100 80GB (80% Faster vs Original)

```
Loading pipeline components...: 100%| 5/5 [00:00<00:00, 5.60it/s]
100%| 100/100 [00:02<00:00, 42.94it/s]
100%| 100/100 [00:01<00:00, 62.68it/s]
100%| 100/100 [00:01<00:00, 64.78it/s]
100%| 100/100 [00:01<00:00, 67.92it/s]
100%| 100/100 [00:01<00:00, 66.79it/s]
```

Source: <https://huggingface.co/segmind/tiny-sd>





# Change backbone

Change runwayml/stable-diffusion-v1-5 for segmind/tiny-sd

```
~/ .cache/huggingface/hub: du models--runwayml--stable-diffusion-v1-5/
```

| # | path                                                                       | apparent | physical |
|---|----------------------------------------------------------------------------|----------|----------|
| 0 | /Users/aldo/.cache/huggingface/hub/models--runwayml--stable-diffusion-v1-5 | 10.9 GB  | 10.9 GB  |

```
~/ .cache/huggingface/hub: du models--segmind--tiny-sd/
```

| # | path                                                        | apparent | physical |
|---|-------------------------------------------------------------|----------|----------|
| 0 | /Users/aldo/.cache/huggingface/hub/models--segmind--tiny-sd | 1.0 GB   | 1.0 GB   |

Model takes 10.9x less space on disk

Source: my laptop

# Change backbone

Here's some example generated images from the new backbone

Pro can still generate high quality images.



Source: <https://huggingface.co/segmind/tiny-sd>

# Changes and training configuration

- Used batch size of 4 because of GPU limitations (otherwise the GPU overheated and the PC crashed.)
- Original batch size was 128. This may mean we at least need 32x the training steps to have “equivalent” results.
- Original training steps were 150000.

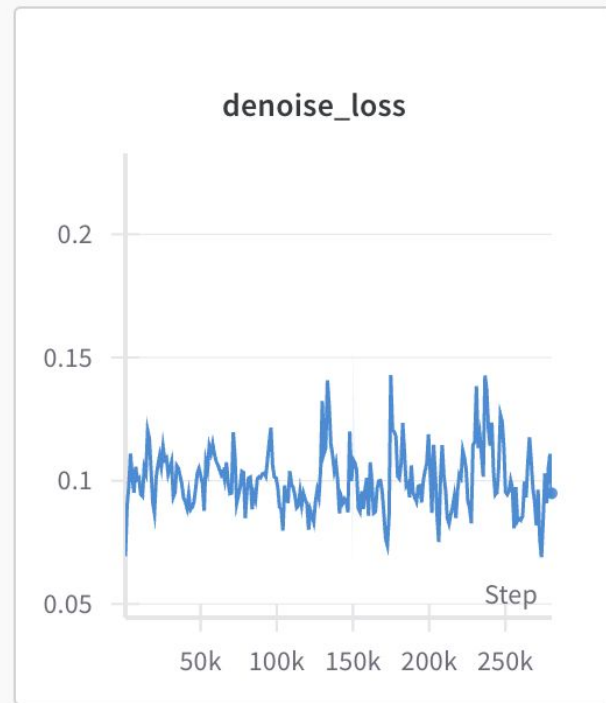
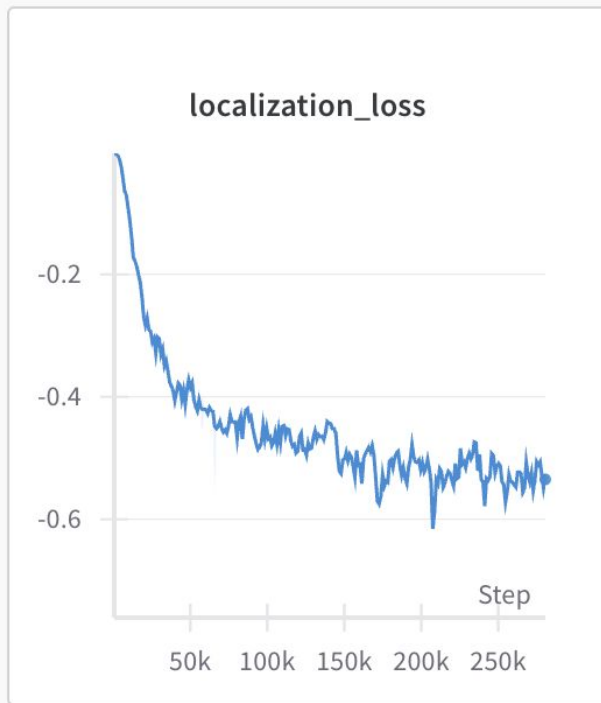
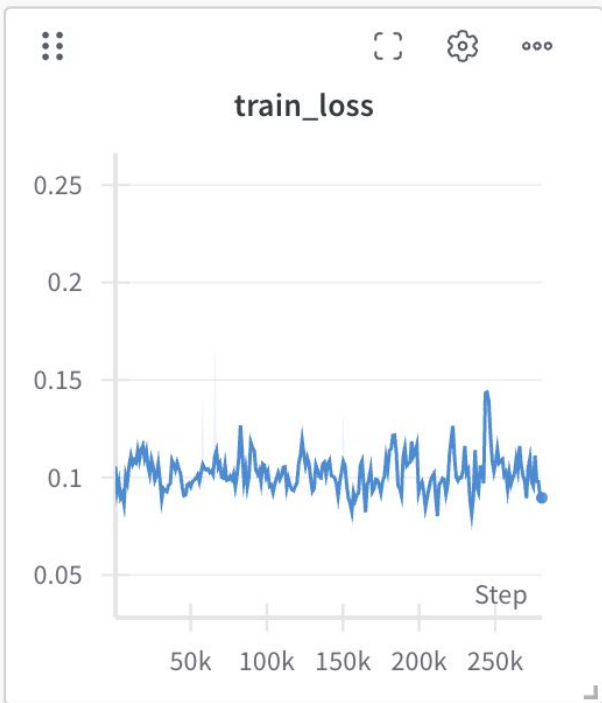
**Source:** FastComposer, Xiao et al (2023)

# Results

# Results

- The model ended up training for 280000 steps over 5 days (started on Thursday 22th, ended on Monday 26th)

# Loss curves



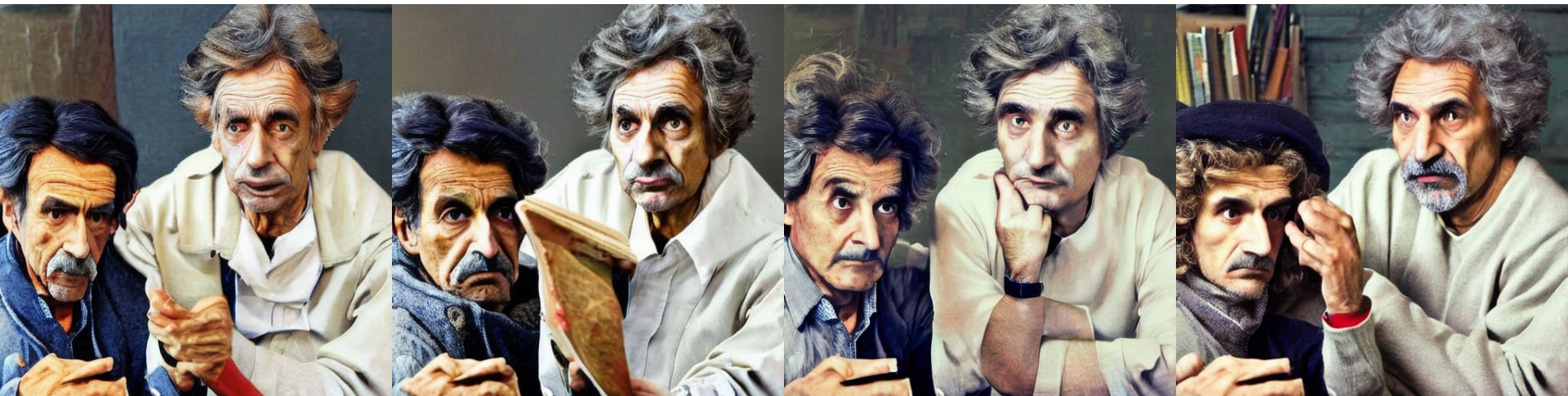
# Images generated throughout training

70k

140k

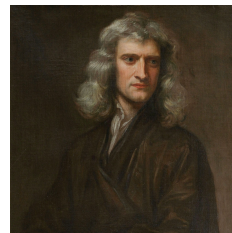
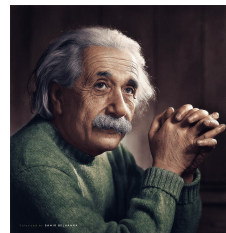
210k

280k



Prompt: a man and a man reading a book

subjects:





# Single-subject model evaluation

After 280k steps:

```
Image Alignment: 0.2949257685703924 +- 0.17447025696402507  
Text Alignment: 0.2322363026936849 +- 0.03457039513999198
```

Trained model provided on repo:

```
Image Alignment: 0.513158406194998 +- 0.18823535881885037  
Text Alignment: 0.22920188903808594 +- 0.03466488543321299
```

With a 32x smaller batch size, training for 1.86x steps, we got:

- 60% of the original trained model **image alignment**
- 100% of the original trained model **text alignment**

# Single-subject model evaluation



It clearly has learned to replicate the facial features, just not at high quality.

**Remember:** these images are completely generated from information embedded on a single token ('man,' in this case)

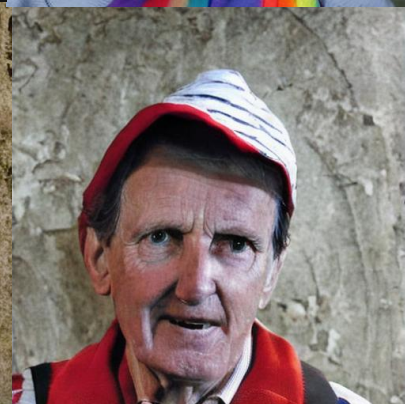
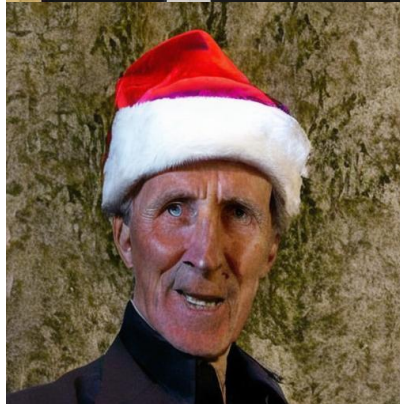


# Single-subject model evaluation



It clearly has learned to replicate the facial features, just not at high quality.

**Remember:** these images are completely generated from information embedded on a single token ('man,' in this case)



# Conclusion

Given more time, I think it's safe to say we can get better results that accurately represent multiple subjects, at a much lower inference time (1.46x) and model size (10.9x). We only trained for 5% (280k/4.8M) of samples compared to the paper-provided model.

Maybe *another approach* could have been to distill the given trained FastComposer model into a smaller architecture (this would have required a new custom training loop for model distillation, instead of training from scratch, as we have done here).

Thank you!